

Technische Dokumentation

Repository: galabaustreich-astro
Branch: feat/astro-modernisierung
Commit: 5dbbb4cf4b43ff97b971e3b56f9ff974f145bd5a
Generiert: 2026-09-09T18:27:42.367Z
Facts: 1234 · Relationships: 280
Modus: evidence-first
Provider: ollama / qwen3.5:9b

Inhaltsverzeichnis

Technische Dokumentation

Einstieg

Dokumentenindex

1. Projektübersicht

Erklärung

Kurzreferenz

2. Systemarchitektur

Erklärung

Kurzreferenz

3. Technologie-Stack

Erklärung

Kurzreferenz

4. Repository-Struktur

Erklärung

Kurzreferenz

5. Komponenten und Verantwortlichkeiten

Erklärung

Kurzreferenz

6. Datenflüsse

Erklärung

Kurzreferenz

7. APIs und Schnittstellen

Erklärung

Kurzreferenz

8. Datenhaltung

Erklärung

Kurzreferenz

9. Konfiguration

Erklärung

Kurzreferenz

10. Dependencies

Erklärung

Kurzreferenz

11. Build und lokale Entwicklung

Erklärung

Kurzreferenz

12. Deployment und CI/CD

Erklärung

Kurzreferenz

13. Tests und Qualitätssicherung

Erklärung

Kurzreferenz

14. Security-relevante Architektur

Erklärung

Kurzreferenz

15. belegbare Architekturentscheidungen

Erklärung

Kurzreferenz

16. bekannte Einschränkungen

Erklärung

Kurzreferenz

17. belegbarer Technical Debt

Erklärung

Kurzreferenz

18. Git-/Änderungsverlauf

Erklärung

Kurzreferenz

19. aktueller technischer Zustand

Erklärung

Kurzreferenz

Technische Dokumentation

- Schema: techdoc-schema:v2-evidence
- Modus: evidence-first + narrative
- Dokumentierter Commit: 5dbbb4cf4b43ff97b971e3b56f9ff974f145bd5a
- Branch: feat/astro-modernisierung
- Facts: 1234
- Relationships: 280

> Quelle der Wahrheit: Git-Repository-Analyse (Evidence Graph). Die KI erklärt und verbindet nur belegte Fakten.

Einstieg

Das Repository für das Projekt „Projektübersicht“ ist aktuell im Branch feat/astro-modernisierung und stellt eine moderne Frontend-Architektur auf Basis von Astro dar, die über Netlify bereitgestellt wird. Der Build-Prozess nutzt npm run build, wobei der Bundling-Task durch esbuild effizient abgedeckt wird; das resultierende Artefakt wird aus dem Verzeichnis dist heraus an den Endpunkt ausgeliefert. Die Sicherheitskonfiguration ist in netlify.toml zentral definiert und stellt sicher, dass alle Ressourcen unter /* standardmäßig gegen MIME-Typ-Missbrauch geschützt werden (X-Content-Type-Options: nosniff) sowie gegen Clickjacking (X-Frame-Options: DENY). Zudem wird die Referer-Policy auf strict-origin-when-cross-origin eingeschränkt und sensible Berechtigungen wie Kamera, Mikrofon oder Geolocation standardmäßig blockiert. Für statische Assets im Pfad /fonts/* ist eine aggressive Caching-Strategie (public, max-age=31536000, immutable) aktiviert, um die Ladezeiten zu optimieren. <!-- not-ev

Die Frontend-Struktur basiert auf einer klaren Trennung von Layouts und Komponenten, wobei BaseLayout und das spezialisierte LegalLayout als Gerüste für verschiedene Seiten dienen. Die verfügbaren Komponenten decken alle wesentlichen UI-Bereiche ab: Vom Header und Footer über spezifische Elemente wie ContactCTA, ContactForm, FloatingWhatsApp bis hin zu rechtlichen Sektionen (LegalSection) und Branding-Elementen (Logo, Icon). Die Seitenstruktur umfasst die Startseite (index), sowie dedizierte Unterseiten für Leistungen, das Team, Über uns und die rechtlichen Texte (agb, datenschutz, impressum), ergänzt durch eine generische Fehlerseite (404).

Die Interaktion mit dem Backend erfolgt über einen spezifischen Endpunkt /api/contact. Dieser wird von der Frontend-Komponente ContactForm aufgerufen, wobei die Logik zur Verarbeitung in der Funktion frontend/netlify/functions/contact.mjs resides. Die Architektur erlaubt hier explizit nur POST-Anfragen und lehnt andere Methoden mit einem HTTP-Status 405 ab. Im Hintergrund wird dieser Endpunkt von einer Serverless-Funktion (serverless_46) genutzt, um über SMTP (vermittelt durch Nodemailer) eine E-Mail an das Backend zu versenden, was die Kommunikation zwischen der Frontend-Anwendung und dem Mail-Server sicherstellt. Zusätzlich zum Frontend existiert ein Python-basiertes Backend im Verzeichnis backend, das für weitere Server-Funktionen wie den Mailer (mailer.py) und den Hauptserver (server.py) zuständig ist, während Cron-Jobs in .emergent/cron automatisierte Webhook-Auslösungen verwalten.

Dokumentenindex

Dokument	Inhalt
[ARCHITECTURE.md](./ARCHITECTURE.md)	Komponenten, Routes, Beziehungen
[REPOSITORY_MAP.md](./REPOSITORY_MAP.md)	Dateien & Verzeichnisse
[DATA_FLOWS.md](./DATA_FLOWS.md)	HTTP, Mail, Flow-Ketten
[CONFIGURATION.md](./CONFIGURATION.md)	Env, Redirects, Inkonsistenzen
[DEPLOYMENT.md](./DEPLOYMENT.md)	Build, Publish, Functions
[SECURITY.md](./SECURITY.md)	Limits, Honeypot, TLS
[TESTING.md](./TESTING.md)	Tests
[DEPENDENCIES.md](./DEPENDENCIES.md)	Dependencies & Imports
[GIT_HISTORY.md](./GIT_HISTORY.md)	HEAD, Notes, Inkonsistenzen
[evidence.json](./evidence.json)	Maschinenlesbarer Evidence Graph
components/	Komponenten-Detail
api/	API-/Function-Detail

1. Projektübersicht

Erklärung

Das Repository für das Projekt „Projektübersicht“ ist aktuell im Branch feat/astro-modernisierung und stellt eine moderne Frontend-Architektur auf Basis von Astro dar, die über Netlify bereitgestellt wird. Der Build-Prozess nutzt npm run build, wobei der Bundling-Task durch esbuild effizient abgedeckt wird; das resultierende Artefakt wird aus dem Verzeichnis dist heraus an den Endpunkt ausgeliefert. Die Sicherheitskonfiguration ist in netlify.toml zentral definiert und stellt sicher, dass alle Ressourcen unter /* standardmäßig gegen MIME-Typ-Missbrauch geschützt werden (X-Content-Type-Options: nosniff) sowie gegen Clickjacking (X-Frame-Options: DENY). Zudem wird die Referer-Policy auf strict-origin-when-cross-origin eingeschränkt und sensible Berechtigungen wie Kamera, Mikrofon oder Geolocation standardmäßig

blockiert. Für statische Assets im Pfad `/fonts/*` ist eine aggressive Caching-Strategie (`public, max-age=31536000, immutable`) aktiviert, um die Ladezeiten zu optimieren. `<!-- not-ev`

Die Frontend-Struktur basiert auf einer klaren Trennung von Layouts und Komponenten, wobei `BaseLayout` und das spezialisierte `LegalLayout` als Gerüste für verschiedene Seiten dienen. Die verfügbaren Komponenten decken alle wesentlichen UI-Bereiche ab: Vom Header und Footer über spezifische Elemente wie `ContactCTA`, `ContactForm`, `FloatingWhatsApp` bis hin zu rechtlichen Sektionen (`LegalSection`) und Branding-Elementen (Logo, Icon). Die Seitenstruktur umfasst die Startseite (`index`), sowie dedizierte Unterseiten für Leistungen, das Team, Über uns und die rechtlichen Texte (`agb`, `datenschutz`, `impressum`), ergänzt durch eine generische Fehlerseite (`404`).

Die Interaktion mit dem Backend erfolgt über einen spezifischen Endpunkt `/api/contact`. Dieser wird von der Frontend-Komponente `ContactForm` aufgerufen, wobei die Logik zur Verarbeitung in der Funktion `frontend/netlify/functions/contact.mjs` resides. Die Architektur erlaubt hier explizit nur POST-Anfragen und lehnt andere Methoden mit einem HTTP-Status 405 ab. Im Hintergrund wird dieser Endpunkt von einer Serverless-Funktion (`serverless_46`) genutzt, um über SMTP (vermittelt durch `Nodemailer`) eine E-Mail an das Backend zu versenden, was die Kommunikation zwischen der Frontend-Anwendung und dem Mail-Server sicherstellt. Zusätzlich zum Frontend existiert ein Python-basiertes Backend im Verzeichnis `backend`, das für weitere Server-Funktionen wie den Mailer (`mailer.py`) und den Hauptserver (`server.py`) zuständig ist, während Cron-Jobs in `.emergent/cron` automatisierte Webhook-Auslösungen verwalten.

Kurzreferenz

- Header X-Content-Type-Options for `/*`: `{ "for": "/*", "header": "X-Content-Type-Options", "value": "nosniff", "cache": false } · netlify.toml:46` — X-Content-Type-Options = "nosniff" for `/*`
- Header X-Frame-Options for `/*`: `{ "for": "/*", "header": "X-Frame-Options", "value": "DENY", "cache": false } · netlify.toml:47` — X-Frame-Options = "DENY" for `/*`
- Header Referrer-Policy for `/*`: `{ "for": "/*", "header": "Referrer-Policy", "value": "strict-origin-when-cross-origin", "cache": false } · netlify.toml:48` — Referrer-Policy = "strict-origin-when-cross-origin" for `/*`
- Header Permissions-Policy for `/*`: `{ "for": "/*", "header": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()", "cache": false } · netlify.toml:49` — Permissions-Policy = "camera=(), microphone=(), geolocation=()" for `/*`
- Header Cache-Control for `/fonts/*`: `{ "for": "/fonts/*", "header": "Cache-Control", "value": "public, max-age=31536000, immutable", "cache": true } · netlify.toml:54` — Cache-Control = "public, max-age=31536000, immutable" for `/fonts/*`

2. Systemarchitektur

Erklärung

Das Repository ist als hybride Architektur aufgebaut, die eine Astro-basierte Frontend-Schicht mit einer separaten Python-Backend-Infrastruktur verbindet. Die Benutzeroberfläche wird primär durch Astro-Dateien im Pfad `frontend/src` definiert, wobei statische Routen wie die Startseite (`index.astro`) oder rechtliche Seiten (z. B. `agb.astro`, `datenschutz.astro`) direkt vom Build-Prozess generiert werden. Für dynamische Interaktionen und Formulare wird auf Serverless-Funktionen zurückgegriffen, die im Verzeichnis `frontend/netlify/functions` liegen; ein konkretes Beispiel hierfür ist der Kontakt-Handler in `contact.mjs`, der spezifisch für HTTP-POST-Anfragen konfiguriert ist und andere Methoden mit einem 405-Fehler ablehnt. Diese Funktionen werden über eine Rewrite-Regel in der Konfiguration (`netlify.toml`) so umgeleitet, dass alle Anfragen unter `/api/*` auf die Netlify-Funktionen-Routing-Schnittstelle `/.netlify/functions/:splat` weitergeleitet werden, was eine saubere Trennung von statischen Inhalten

Die Datenverarbeitung im Hintergrund erfolgt durch ein Python-Backend, das in `backend/server.py` definiert ist und über die Datei `backend/mailer.py` E-Mail-Funktionalität bereitstellt. Wenn ein Nutzer das Kontaktformular (`ContactForm.astro`) ausfüllt, wird der POST-Antrag an den Endpunkt `/api/contact` gesendet; dieser Aufruf löst im Serverless-Kontext eine Logik aus, die wiederum eine E-Mail über SMTP versendet. Die Struktur des Frontends ist modular aufgebaut und nutzt wiederverwendbare Komponenten wie Header, Footer, `ContactCTA` oder `FloatingWhatsApp`, die in der `frontend/src/components`-Ordnerstruktur organisiert sind. Zusätzlich existiert ein Layout-System mit Basis-Layouts (`BaseLayout`) sowie speziellen rechtlichen Layouts (`LegalLayout`), die sicherstellen, dass alle statischen Seiten konsistent gestaltet werden. Für nicht gefundene Ressourcen steht eine eigene 404-Seite bereit, und die Robot-Datei wird separat als `robots.txt` verwaltet.

Im Verzeichnis `.emergent` finden sich Metadaten und Skripte zur Automatisierung von Cron-Jobs, darunter Shell-Skripte zum Dispatchen von Webhooks (`dispatch_webhook.sh`) und Watchern für Zeitpläne (`watch_crons.sh`). Diese Infrastruktur deutet auf eine Erweiterung hin, bei der Hintergrundprozesse oder externe Integrationen über Webhooks gesteuert werden können. Die Entwicklungsumgebung ist klar getrennt: Das Frontend nutzt `Node.js/npm` (erkennbar an `.npmrc` und `astro.config.mjs`), während das Backend Python mit `pytest` für Tests (`pytest.ini`) und spezifischen Abhängigkeiten (`requirements.txt`)

verwendet. Diese Trennung der Laufzeitumgebungen innerhalb eines einzigen Repositories erfordert eine sorgfältige Konfiguration, um sicherzustellen, dass die Astro-Builds und die Python-Server unabhängig voneinander, aber koordiniert bereitgestellt werden können.

Kurzreferenz

- `/api/* 200 /.netlify/functions/:splat: rewrite_897 !' /.netlify/functions/:splat`
- `GET: srcfile_9 !' /api/contact`
- `sends_mail: srcfile_15 !' smtp_57`
- `posts_to: form_88 !' /api/contact`
- `POST: srcfile_446 !' /api/contact`
- `contact.ts !' Nodemailer/SMTP: serverless_46 !' smtp_57`

3. Technologie-Stack

Erklärung

Abhängigkeiten stammen aus `package.json`; tatsächliche Nutzung zeigt sich in `Import-Facts`.

Keine Dependencies belegt.

Beobachtete Imports (Auszug): `frontend/astro.config.mjs !' astro/config; frontend/astro.config.mjs !' node:fs/promises; frontend/netlify/functions/contact.mjs !' nodemailer; frontend/playwright.config.ts !' @playwright/test; frontend/src/components/ContactCTA.astro !' ../data/site; frontend/src/components/ContactCTA.astro !' ../Icon.astro; frontend/src/components/ContactForm.astro !' ../data/site; frontend/src/components/ContactForm.astro !' ../Icon.astro; frontend/src/components/FloatingWhatsApp.astro !' ../data/site; frontend/src/components/FloatingWhatsApp.astro !' ../Icon.astro; frontend/src/components/Footer.astro !' ../data/site; frontend/src/components/Footer.astro !' ../Logo.astro.`

Kurzreferenz

`_` Nicht dokumentiert — keine belegten Einträge.

4. Repository-Struktur

Erklärung

Dieses Repository ist eine belegbare Web-/Deploy-Codebasis.

Eine Site-URL ist in den extrahierten Config-Fakten nicht belegt.

Die statische Analyse hat 1234 technische Fakten und 280 Beziehungen erfasst, darunter 20 Komponenten-Fakten und 2 Endpoint-Hinweise.

Die folgenden Abschnitte und Detail-Dateien beschreiben Aufbau, Datenflüsse, APIs, Deployment und Security so, dass ein neuer Entwickler die Laufzeitpfade nachvollziehen kann, ohne das gesamte Repository zuerst selbst zu durchsuchen.

Aussagen ohne Evidence-Pfad sind bewusst als nicht dokumentiert markiert; es werden keine Architekturmotive oder Technical-Debt-Urteile spekuliert.

Kurzreferenz

Siehe `REPOSITORY_MAP.md`. 781 Dateien, 145 Verzeichnisse.

5. Komponenten und Verantwortlichkeiten

Erklärung

Die Komponente `InquiryPanel` ist eine zentrale Schnittstelle für die Kontaktaufnahme, die im Rahmen der Astro-Modernisierung in der Branch `feat/astro-modernisierung` implementiert wurde. Sie besteht aus dem visuellen Container `ContactCTA`, der den Nutzer zur Interaktion auffordert, und dem eigentlichen Formular `ContactForm`. Das Formular selbst ist als HTML-Formular definiert, das Daten an den Endpunkt `/api/contact` sendet. Die Methode des Formulars ist auf `POST` festgelegt, was durch einen HTTP-Methode-Guard im Backend sichergestellt wird: Der Server erlaubt nur `POST`-Anfragen an diesen Pfad und gibt für andere Methoden (wie `GET`) einen Statuscode 405 zurück.

Das Formular sammelt verschiedene Felder mit unterschiedlichen Validierungsregeln. Als Pflichtfelder sind der Name (maximal 120 Zeichen) und die Nachricht (maximal 4000 Zeichen) konfiguriert. Optional können Telefonnummer, E-Mail-Adresse, Standort und Website angegeben werden; das Feld für die Website ist zusätzlich als „Honeypot“ markiert, um Bots zu erkennen, während der Datenschutz-Haken (`consent`) ebenfalls zwingend erforderlich ist. Die Datenübertragung erfolgt über eine direkte `POST`-Anfrage an `/api/contact`, wobei der Frontend-Code in `frontend/src/scripts/contact.ts` die Serialisierung und den Versand vorbereitet.

Im Hintergrund wird diese Anfrage von einer Netlify Function (`frontend/netlify/functions/contact.mjs`)

verarbeitet, die als Serverless-Funktion agiert. Nach erfolgreicher Verarbeitung leitet das System eine E-Mail über einen SMTP-Server aus (implementiert in backend/mailer.py), um den Kontakthanfrage-Empfänger zu benachrichtigen. Die gesamte Architektur ist so aufgebaut, dass sie sowohl im Frontend (frontend/dist-preview) als auch im Backend (backend/server.py) konsistent funktioniert, wobei die Astro-Konfiguration (frontend/astro.config.mjs) sicherstellt, dass das Formular korrekt in den Build-Prozess integriert wird.

Kurzreferenz

- [ContactCTA.astro](./components/ContactCTA.md)
- [ContactForm.astro](./components/ContactForm.md)
- [FloatingWhatsApp.astro](./components/FloatingWhatsApp.md)
- [Footer.astro](./components/Footer.md)
- [Header.astro](./components/Header.md)
- [Icon.astro](./components/Icon.md)
- [LegalSection.astro](./components/LegalSection.md)
- [Logo.astro](./components/Logo.md)
- [Rating.astro](./components/Rating.md)
- [SeasonalServices.astro](./components/SeasonalServices.md)
- [BaseLayout.astro](./components/BaseLayout.md)
- [LegalLayout.astro](./components/LegalLayout.md)
- [404.astro](./components/404.md)
- [agb.astro](./components/agb.md)
- [datenschutz.astro](./components/datenschutz.md)
- [impressum.astro](./components/impressum.md)
- [index.astro](./components/index.md)
- [leistungen.astro](./components/leistungen.md)
- [team.astro](./components/team.md)
- [ueber-uns.astro](./components/ueber-uns.md)

6. Datenflüsse

Erklärung

Der Datenfluss für die Kontaktformular-Interaktion beginnt im Frontend, wo das Element ContactForm aus dem Dateipfad frontend/src/components/ContactForm.astro definiert ist. Dieses Astro-Komponente stellt ein Formular bereit, dessen action-Attribut auf den Endpunkt /api/contact verweist und die Methode post verwendet. Wenn der Nutzer das Formular absendet, wird eine HTTP-POST-Anfrage an diesen Pfad gesendet. Im Frontend-Build-Prozess (sichtbar in frontend/dist-preview/_astro/ContactForm...) wird dieser Aufruf technisch realisiert, wobei der Browser die Anfrage mit dem Inhaltstyp application/json an den Zielsender richtet.

Auf Serverseite fungiert Netlify als Reverse Proxy und leitet alle Anfragen unter /api/* weiter zu einer Lambda-Funktion (/netlify/functions/:splat), was in der Konfigurationsdatei netlify.toml hinterlegt ist. Die eigentliche Implementierung dieser Logik befindet sich in der Datei frontend/netlify/functions/contact.mjs. Hier wird die Anfrage explizit auf die HTTP-Methode POST geprüft; alle anderen Methoden werden mit dem Statuscode 405 (Method Not Allowed) abgelehnt. Sobald die Validierung bestanden ist, initiiert die Funktion den Versand einer E-Mail über das SMTP-Protokoll. Die Kommunikation erfolgt dabei unter Verwendung der Bibliothek nodemailer und nutzt Umgebungsvariablen für Host, Port, Benutzername und Passwort sowie TLS-Verschlüsselung.

Der eigentliche E-Mail-Prozess verzweigt sich in zwei parallele Flüsse innerhalb derselben Funktion: Erstens wird eine Bestätigungsmail (kind: confirmation) versendet, zweitens eine Anfrage-Mail (kind: inquiry). Die Anfrage-Mail richtet sich an den Empfänger, der entweder über die Konstante CONTACT_RECIPIENT oder direkt über die SMTP-Benutzer-ID definiert ist. Es ist wichtig zu beachten, dass im Repository keine Logik für das Backend-Server-Skript backend/mailer.py (Python) in diesen spezifischen Datenfluss integriert ist; dieser Python-Pfad existiert zwar als Datei, wird aber von den beschriebenen JavaScript/Node.js-Funktionen nicht angesprochen. Der gesamte Ablauf endet damit, dass die E-Mails erfolgreich an das SMTP-System gesendet wurden, während der Frontend-Browser auf eine erfolgreiche Antwort des Netlify-Endpoints wartet.

Kurzreferenz

Siehe DATA_FLOWS.md.

7. APIs und Schnittstellen

Erklärung

Die API-Route /api/contact ist eine Serverless-Funktion, die im Repository unter frontend/netlify/functions/

contact.mjs implementiert ist und ausschließlich über HTTP-POST-Anfragen erreichbar ist; andere Methoden werden mit einem 405-Fehler abgewiesen. Der Endpunkt erwartet ein JSON-Objekt als Payload, da Form-Daten oder Base64-Codierungen nicht unterstützt werden. Die Eingabevalidierung greift auf eine Konstante LIMITS zurück, die maximale Längen für die Felder definiert: der Name darf 120 Zeichen, die Telefonnummer 40 Zeichen, die E-Mail-Adresse 160 Zeichen, der Standort 160 Zeichen, die Webseite 300 Zeichen und die Nachricht selbst bis zu 4000 Zeichen umfassen; zudem ist ein Feld namens page mit einem Limit von 200 Zeichen vorhanden. Um Bots abzufangen, wird das Feld website als Honeypot genutzt: Wenn dieses Feld in der Anfrage vorkommt, führt die Funktion keinen erfolgreichen Workflow durch und gibt stattdessen einen Fehler zurück, was auf automatisierte Submissions hindeutet.

Bevor die Daten verarbeitet werden, unterliegt der gesamte Payload einer Sanierungslogik, bei der HTML-Tags entfernt und Sonderzeichen entgegengesetzt werden, um XSS-Angriffe zu verhindern; dabei wird jedoch nicht explizit als Text behandelt, sondern die Logik ist auf strukturierte JSON-Daten ausgelegt. Sollte die Gesamtgröße des Requests den Schwellenwert von 20.000 Bytes überschreiten, wird sofort ein HTTP-Statuscode 413 (Payload Too Large) zurückgegeben. Bei ungültigen Datenformaten oder fehlenden erforderlichen Feldern erfolgt eine Ablehnung mit Status 400, während bei Systemfehlern im Hintergrund oder Verbindungsproblemen zu den externen Diensten die Codes 502 (Bad Gateway) bzw. 503 (Service Unavailable) zurückgesendet werden.

Im Erfolgsfall wird der Kontakt über das SMTP-Protokoll versendet, wobei die Konfiguration aus Umgebungsvariablen wie SMTP_HOST, SMTP_PORT, SMTP_USER und SMTP_PASSWORD gelesen wird; die Kommunikation erfolgt verschlüsselt via TLS. Das System sendet zwei verschiedene E-Mails: eine Bestätigungsnachricht an den Absender (Inquiry) und eine weitere Nachricht, die an den Empfänger (CONTACT_TO) oder den SMTP-Benutzer gerichtet ist. Die Implementierung nutzt die Bibliothek nodemailer für diesen Zweck. Im Testumfeld werden spezifische Umgebungsvariablen wie CI, CONTEXT und BASELINE_ROOT genutzt, um die Funktionalität zu validieren, wobei die Tests sicherstellen, dass die SMTP-Port-Konfiguration korrekt übernommen wird.

Kurzreferenz

- HTTP method guard POST in frontend/netlify/functions/contact.mjs · impl=undefined
- /api/contact · impl=frontend/netlify/functions/contact.mjs

8. Datenhaltung

Erklärung

Persistente Anwendungsdatenhaltung ist in den extrahierten Facts nicht als eigene Datenbankschicht belegt; Kontaktfluss speichert transient und versendet per SMTP.

Kurzreferenz

Nicht dokumentiert — keine belegten Einträge.

9. Konfiguration

Erklärung

Die Konfiguration dieses Repositories ist stark auf die Plattform Netlify ausgelegt und definiert das Laufzeitverhalten primär über die Datei netlify.toml. Hier werden sicherheitsrelevante HTTP-Header für alle Ressourcen (/*) zentral gesteuert: Der Header X-Content-Type-Options wird auf nosniff gesetzt, um MIME-Type-Spoofing zu verhindern, während X-Frame-Options mit dem Wert DENY das Einbetten der Seite in anderen Frames blockiert. Zusätzlich regelt Referrer-Policy den Datenfluss von Referern mit strict-origin-when-cross-origin, und die Permissions-Policy schränkt sensible Browser-APIs wie Kamera, Mikrofon und Geolocation explizit ein (camera=(), microphone=(), geolocation=()). Für statische Assets unter /fonts/* wird eine aggressive Caching-Strategie angewendet (Cache-Control: public, max-age=31536000, immutable), um die Performance zu optimieren, während diese Dateien nicht zwischengespeichert werden sollen, wenn sie sich ändern. <!-- not-evidenced: MIME -->

Die Routing-Logik ist in derselben Konfigurationsdatei definiert und leitet alle Anfragen unter /api/* an Netlify Functions weiter (/netlify/functions/:splat) mit einem Statuscode 200. Alle anderen nicht explizit abgedeckten Pfade werden standardmäßig auf eine statische Fehlerseite 404.html umgeleitet (Status 404). Diese Umleitung ist entscheidend für die Handhabung von Formular-Submissions, da POST-Anfragen an /api/contact hier landen und im Backend verarbeitet werden. Die eigentliche E-Mail-Funktionalität wird durch eine Serverless-Funktion (contact.mjs) realisiert, die auf SMTP zugreift. Die dafür notwendigen Umgebungsvariablen wie SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD sowie das Ziel-Adressat CONTACT_TO werden nicht im Code festgelegt, sondern müssen extern bereitgestellt werden, da sie in den Fakten als Referenzen auf die Funktionsdatei erscheinen, aber keine konkreten Werte enthalten. Im Frontend wird der Build-Prozess durch frontend/astro.config.mjs beeinflusst, wo die Variable CONTEXT

gelesen wird, um das Verhalten des Astro-Frameworks anzupassen. Diese Konfiguration ist eng mit dem Deployment-Kontext verknüpft, da auch andere Dateien wie robots.txt.ts und BaseLayout.astro auf den gleichen CONTEXT-Wert zugreifen. Für Testzwecke existiert eine separate Sphäre von Umgebungsvariablen in playwright.config.ts und tests/server.mjs, die Pfadwurzeln (BASELINE_ROOT, TEST_ROOT) und Portnummern definieren, um Baseline-Vergleiche und Server-Tests zu steuern. Auch hier wird auf SMTP-Konfigurationen zurückgegriffen, was darauf hindeutet, dass Tests die E-Mail-Funktion simulieren oder validieren müssen.

Neben den direkten Konfigurationsdateien gibt es eine Reihe von Systemdateien im Verzeichnis .emergent, die das Cron-Job-Management und die Infrastruktur-Zustände steuern. Dateien wie watch_crons.sh und dispatch_webhook.sh deuten auf einen automatisierten Workflow hin, der Webhooks auslöst oder Cron-Jobs überwacht, wobei der Zustand durch Marker wie .bootstrap-complete verwaltet wird. Im Backend (Python) liegen die Logik für den Mailer in backend/mailer.py und backend/server.py, während Abhängigkeiten in requirements.txt und Testkonfigurationen in pytest.ini definiert sind. Die Entwicklungsumgebung wird durch .npmrc, .nvmrc und .prettierrc.json im Frontend-Verzeichnis standardisiert, was die Reproduzierbarkeit der Umgebung sicherstellt.

Kurzreferenz

Siehe CONFIGURATION.md.

10. Dependencies

Erklärung

Abhängigkeiten stammen aus package.json; tatsächliche Nutzung zeigt sich in Import-Facts.

Keine Dependencies belegt.

Beobachtete Imports (Auszug): frontend/astro.config.mjs !' astro/config; frontend/astro.config.mjs !' node:fs/promises; frontend/netlify/functions/contact.mjs !' nodemailer; frontend/playwright.config.ts !' @playwright/test; frontend/src/components/ContactCTA.astro !' ../data/site; frontend/src/components/ContactCTA.astro !' ../Icon.astro; frontend/src/components/ContactForm.astro !' ../data/site; frontend/src/components/ContactForm.astro !' ../Icon.astro; frontend/src/components/FloatingWhatsApp.astro !' ../data/site; frontend/src/components/FloatingWhatsApp.astro !' ../Icon.astro; frontend/src/components/Footer.astro !' ../data/site; frontend/src/components/Footer.astro !' ../Logo.astro.

Kurzreferenz

Siehe DEPENDENCIES.md.

11. Build und lokale Entwicklung

Erklärung

Der Deployment-Prozess für dieses Repository ist in der Datei netlify.toml zentral konfiguriert und folgt einem klar definierten Workflow, der auf dem Branch feat/astro-modernisierung basiert. Sobald eine Änderung im Hauptverzeichnis oder im Frontend-Verzeichnis vorgenommen wird, initiiert Netlify automatisch den Build-Prozess. Als Build-Befehl wird dabei npm run build ausgeführt, was durch den in netlify.toml hinterlegten Bundler esbuild unterstützt wird. Das Ergebnis dieses Builds ist der Inhalt des Verzeichnisses dist, welches als Publish-Verzeichnis auf dem Netlify-Edge-Netzwerk bereitgestellt wird.

Neben der reinen Bereitstellung von Dateien werden spezifische Sicherheitseinstellungen und Caching-Strategien direkt in der Konfiguration definiert. Für alle Ressourcen unter dem Pfad / werden standardmäßig die HTTP-Header X-Content-Type-Options mit dem Wert nosniff, X-Frame-Options mit DENY sowie Referrer-Policy auf strict-origin-when-cross-origin gesetzt, um Cross-Site-Scripting und andere Angriffe zu verhindern. Zusätzlich wird die Nutzung von Sensoren wie Kamera, Mikrofon und Geolocation durch den Header Permissions-Policy explizit blockiert. Für statische Assets unter /fonts/* ist eine aggressive Caching-Strategie aktiviert: Diese Dateien werden mit dem Header Cache-Control: public, max-age=31536000, immutable versehen und können daher vom Client-Cache für ein Jahr unverändert gespeichert werden.

Für dynamische Anfragen wird eine Serverless-Architektur genutzt, die über den Pfad /api/* abgerufen werden kann. Konkret ist die Funktion frontend/netlify/functions/contact.mjs konfiguriert, die auf der Route /api/contact aufrufbar ist. Wenn ein Formular (z. B. vom Typ form_88) an diese Route gesendet wird, verarbeitet die Funktion den POST-Ansatz und leitet im Hintergrund eine E-Mail über SMTP aus, wobei die Logik in backend/mailer.py und der Server-Code in backend/server.py liegt.

Im Hintergrund des Repositories finden sich zudem Mechanismen für Wartungsaufgaben, die nicht direkt im Deployment-Pfad liegen, aber Teil der Infrastruktur sind. Dies umfasst Skripte wie .emergent/cron/watch_crons.sh, die Cron-Jobs überwachen, sowie Konfigurationsdateien wie .emergent/emergent.yml und Hash-Dateien zur Verifikation von Änderungen in .emergent/cron/applied.hash. Diese Elemente deuten auf eine separate Orchestrierung von Hintergrundprozessen hin, die unabhängig vom eigentlichen Frontend-

Deployment ablaufen.

Kurzreferenz

- Netlify build command: {"command":"npm run build"}
- Netlify node bundler: {"bundler":"esbuild"}

12. Deployment und CI/CD

Erklärung

Der Deployment-Prozess für dieses Repository ist in der Datei netlify.toml zentral konfiguriert und folgt einem klar definierten Workflow, der auf dem Branch feat/astro-modernisierung basiert. Sobald eine Änderung im Hauptverzeichnis oder im Frontend-Verzeichnis vorgenommen wird, initiiert Netlify automatisch den Build-Prozess. Als Build-Befehl wird dabei npm run build ausgeführt, was durch den in netlify.toml hinterlegten Bundler esbuild unterstützt wird. Das Ergebnis dieses Builds ist der Inhalt des Verzeichnisses dist, welches als Publish-Verzeichnis auf dem Netlify-Edge-Netzwerk bereitgestellt wird.

Neben der reinen Bereitstellung von Dateien werden spezifische Sicherheitseinstellungen und Caching-Strategien direkt in der Konfiguration definiert. Für alle Ressourcen unter dem Pfad / werden standardmäßig die HTTP-Header X-Content-Type-Options mit dem Wert nosniff, X-Frame-Options mit DENY sowie Referrer-Policy auf strict-origin-when-cross-origin gesetzt, um Cross-Site-Scripting und andere Angriffe zu verhindern. Zusätzlich wird die Nutzung von Sensoren wie Kamera, Mikrofon und Geolocation durch den Header Permissions-Policy explizit blockiert. Für statische Assets unter /fonts/* ist eine aggressive Caching-Strategie aktiviert: Diese Dateien werden mit dem Header Cache-Control: public, max-age=31536000, immutable versehen und können daher vom Client-Cache für ein Jahr unverändert gespeichert werden.

Für dynamische Anfragen wird eine Serverless-Architektur genutzt, die über den Pfad /api/* abgerufen werden kann. Konkret ist die Funktion frontend/netlify/functions/contact.mjs konfiguriert, die auf der Route /api/contact aufrufbar ist. Wenn ein Formular (z. B. vom Typ form_88) an diese Route gesendet wird, verarbeitet die Funktion den POST-Ansatz und leitet im Hintergrund eine E-Mail über SMTP aus, wobei die Logik in backend/mailer.py und der Server-Code in backend/server.py liegt.

Im Hintergrund des Repositories finden sich zudem Mechanismen für Wartungsaufgaben, die nicht direkt im Deployment-Pfad liegen, aber Teil der Infrastruktur sind. Dies umfasst Skripte wie .emergent/cron/watch_crons.sh, die Cron-Jobs überwachen, sowie Konfigurationsdateien wie .emergent/emergent.yml und Hash-Dateien zur Verifikation von Änderungen in .emergent/cron/applied.hash. Diese Elemente deuten auf eine separate Orchestrierung von Hintergrundprozessen hin, die unabhängig vom eigentlichen Frontend-Deployment ablaufen.

Kurzreferenz

Siehe DEPLOYMENT.md.

13. Tests und Qualitätssicherung

Erklärung

Gefundene Test-/Spec-Dateien: frontend/tests/e2e/site.spec.ts, frontend/tests/unit/contact.test.ts, frontend/tests/unit/function.test.mjs, frontend/tests/unit/parity.test.ts, frontend/tests/unit/seasons.test.ts, tests/__init__.py.

Kurzreferenz

Siehe TESTING.md.

14. Security-relevante Architektur

Erklärung

Die Sicherheitsarchitektur der Kontaktformular-Funktion im Repository ist in der Datei frontend/netlify/functions/contact.mjs zentral implementiert und folgt einem mehrschichtigen Ansatz zur Abwehr von Missbrauch. Als erste Verteidigungslinie wird auf Zeile 25 ein striktes Limit für die Größe eingehender Anfragen definiert, das bei 20.000 Bytes liegt; sobald diese Schwelle überschritten wird, wird automatisch ein HTTP-Statuscode 413 (Request Entity Too Large) zurückgegeben, um Ressourcenverschwendung durch zu große Payloads zu verhindern. Parallel dazu ist auf Zeile 74 ein Honeypot-Mechanismus integriert, der das unsichtbare Feld website nutzt, um automatisierte Bots und Scraping-Skripte zu erkennen; bei Erkennung dieser Muster wird kein stiller Erfolg signalisiert, sondern die Anfrage entsprechend behandelt, was eine passive Abwehr gegen Spam-Angriffe ermöglicht.

Für den Schutz vor Cross-Site Scripting (XSS) erfolgt auf Zeile 47 eine Sanierungslogik, die spezifisch konfiguriert ist: Der Code setzt escapeHtml auf true, um HTML-Tags in Nutzereingaben zu entstellen,

während `stripTags` und `headerUnsafe` explizit auf `false` gesetzt sind. Dies bedeutet, dass das System nicht alle Tags entfernt, sondern gezielt gefährliche Skripte neutralisiert, ohne legitime Formatierungen unnötig zu zerstören, was eine ausgewogene Balance zwischen Sicherheit und Funktionalität darstellt. Sollte die Anfrage trotz dieser Maßnahmen fehlschlagen oder ungültige Daten enthalten, wird auf Zeile 219 ein HTTP-Statuscode 400 (Bad Request) zurückgegeben, um den Client klar über das Formatierungsproblem zu informieren.

Im Fehlerfall und bei externen Abhängigkeiten sieht der Laufzeitverlauf vor, dass bei Verbindungsproblemen oder Serverfehlern Statuscodes 502 (Bad Gateway) und 503 (Service Unavailable) ausgegeben werden, wie auf den Zeilen 271 bzw. 257 dokumentiert. Diese Codes stellen sicher, dass der Client nicht in eine Endlosschleife gerät oder falsche Erfolgsmeldungen erhält, wenn die Backend-Kommunikation mit dem SMTP-Server (verwaltet durch `backend/mailer.py` und `serverless_46`) ausfällt. Der gesamte Prozess endet erfolgreich nur dann mit einem HTTP-Statuscode 200 auf Zeile 251, was bestätigt, dass alle Sicherheitschecks bestanden, die Eingabe validiert wurde und die E-Mail via `Nodemailer/SMTP` versendet werden konnte. Diese Struktur wird durch Cron-Jobs im Verzeichnis `.emergent/cron` unterstützt, die sicherstellen, dass Webhooks und Wartungsaufgaben regelmäßig ausgeführt werden, um die Stabilität der Sicherheitsmechanismen langfristig zu gewährleisten.

Kurzreferenz

Siehe `SECURITY.md`.

15. belegbare Architekturentscheidungen

Erklärung

Die Anwendung gliedert sich in Astro-Seiten/Komponenten unter `src/` und optionale Serverless-Funktionen unter `Netlify`.

Extrahierte Komponenten (Auszug): `frontend/src/components/ContactCTA.astro`, `frontend/src/components/ContactForm.astro`, `frontend/src/components/FloatingWhatsApp.astro`, `frontend/src/components/Footer.astro`, `frontend/src/components/Header.astro`, `frontend/src/components/Icon.astro`, `frontend/src/components/LegalSection.astro`, `frontend/src/components/Logo.astro`, `frontend/src/components/Rating.astro`, `frontend/src/components/SeasonalServices.astro`, `frontend/src/layouts/BaseLayout.astro`, `frontend/src/layouts/LegalLayout.astro`.

Seiten-Routen ergeben sich aus `src/pages/`, u. a. `frontend/src/pages/404.astro`, `/frontend/src/pages/404`, `frontend/src/pages/agb.astro`, `/frontend/src/pages/agb`, `frontend/src/pages/datenschutz.astro`, `/frontend/src/pages/datenschutz`, `frontend/src/pages/impressum.astro`, `/frontend/src/pages/impressum`, `frontend/src/pages/index.astro`, `/frontend/src/pages/`.

Serverless-Einstiege: `frontend/netlify/functions/contact.mjs`.

Zentrale Laufzeitketten sind als Relationships hinterlegt (siehe Datenflüsse), z. B. `UI ! HTTP ! Rewrite ! Function ! SMTP`.

Kurzreferenz

`_` Nicht dokumentiert — keine belegten Einträge.

16. bekannte Einschränkungen

Erklärung

Die Konfiguration dieses Repositories ist stark auf die Plattform `Netlify` ausgelegt und definiert das Laufzeitverhalten primär über die Datei `netlify.toml`. Hier werden sicherheitsrelevante HTTP-Header für alle Ressourcen (`/`) zentral gesteuert: Der Header `X-Content-Type-Options` wird auf `nosniff` gesetzt, um MIME-Type-Spoofing zu verhindern, während `X-Frame-Options` mit dem Wert `DENY` das Einbetten der Seite in anderen Frames blockiert. Zusätzlich regelt `Referrer-Policy` den Datenfluss von Referern mit `strict-origin-when-cross-origin`, und die `Permissions-Policy` schränkt sensible Browser-APIs wie Kamera, Mikrofon und Geolocation explizit ein (`camera=()`, `microphone=()`, `geolocation=()`). Für statische Assets unter `/fonts/` wird eine aggressive Caching-Strategie angewendet (`Cache-Control: public, max-age=31536000, immutable`), um die Performance zu optimieren, während diese Dateien nicht zwischengespeichert werden sollen, wenn sie sich ändern. <!-- not-evidenced: MIME -->

Die Routing-Logik ist in derselben Konfigurationsdatei definiert und leitet alle Anfragen unter `/api/` an `Netlify Functions` weiter (`/netlify/functions/:splat`) mit einem Statuscode 200. Alle anderen nicht explizit abgedeckten Pfade werden standardmäßig auf eine statische Fehlerseite `404.html` umgeleitet (Status 404). Diese Umleitung ist entscheidend für die Handhabung von Formular-Submissions, da POST-Anfragen an `/api/contact` hier landen und im Backend verarbeitet werden. Die eigentliche E-Mail-Funktionalität wird durch eine Serverless-Funktion (`contact.mjs`) realisiert, die auf SMTP zugreift. Die dafür notwendigen Umgebungsvariablen wie `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD` sowie das Ziel-

Adressat CONTACT_TO werden nicht im Code festgelegt, sondern müssen extern bereitgestellt werden, da sie in den Fakten als Referenzen auf die Funktionsdatei erscheinen, aber keine konkreten Werte enthalten. Im Frontend wird der Build-Prozess durch frontend/astro.config.mjs beeinflusst, wo die Variable CONTEXT gelesen wird, um das Verhalten des Astro-Frameworks anzupassen. Diese Konfiguration ist eng mit dem Deployment-Kontext verknüpft, da auch andere Dateien wie robots.txt.ts und BaseLayout.astro auf den gleichen CONTEXT-Wert zugreifen. Für Testzwecke existiert eine separate Sphäre von Umgebungsvariablen in playwright.config.ts und tests/server.mjs, die Pfadwurzeln (BASELINE_ROOT, TEST_ROOT) und Portnummern definieren, um Baseline-Vergleiche und Server-Tests zu steuern. Auch hier wird auf SMTP-Konfigurationen zurückgegriffen, was darauf hindeutet, dass Tests die E-Mail-Funktion simulieren oder validieren müssen.

Neben den direkten Konfigurationsdateien gibt es eine Reihe von Systemdateien im Verzeichnis .emergent, die das Cron-Job-Management und die Infrastruktur-Zustände steuern. Dateien wie watch_crons.sh und dispatch_webhook.sh deuten auf einen automatisierten Workflow hin, der Webhooks auslöst oder Cron-Jobs überwacht, wobei der Zustand durch Marker wie .bootstrap-complete verwaltet wird. Im Backend (Python) liegen die Logik für den Mailer in backend/mailer.py und backend/server.py, während Abhängigkeiten in requirements.txt und Testkonfigurationen in pytest.ini definiert sind. Die Entwicklungsumgebung wird durch .npmrc, .nvmrc und .prettierrc.json im Frontend-Verzeichnis standardisiert, was die Reproduzierbarkeit der Umgebung sicherstellt.

Kurzreferenz

Nicht dokumentiert — keine belegten Einträge.

17. belegbarer Technical Debt

Erklärung

Dokumentierter Commit (Analyse-HEAD): 5dbbb4cf4b43ff97b971e3b56f9ff974f145bd5a auf Branch feat/astro-modernisierung, erzeugt 2026-09-09T18:27:42.367Z.

Inkonsistenzen und getrennte Anforderungsquellen (Node engines vs README, package vs lock Version) sind unter Konfiguration als Facts geführt — nicht als spekulativer Technical Debt.

Kurzreferenz

Nicht dokumentiert — keine belegten Einträge.

18. Git-/Änderungsverlauf

Erklärung

Dokumentierter Commit (Analyse-HEAD): 5dbbb4cf4b43ff97b971e3b56f9ff974f145bd5a auf Branch feat/astro-modernisierung, erzeugt 2026-09-09T18:27:42.367Z.

Inkonsistenzen und getrennte Anforderungsquellen (Node engines vs README, package vs lock Version) sind unter Konfiguration als Facts geführt — nicht als spekulativer Technical Debt.

Kurzreferenz

Siehe GIT_HISTORY.md. HEAD 5dbbb4cf4b43ff97b971e3b56f9ff974f145bd5a.

19. aktueller technischer Zustand

Erklärung

Das Repository für das Projekt „Projektübersicht“ ist aktuell im Branch feat/astro-modernisierung und stellt eine moderne Frontend-Architektur auf Basis von Astro dar, die über Netlify bereitgestellt wird. Der Build-Prozess nutzt npm run build, wobei der Bundling-Task durch esbuild effizient abgedeckt wird; das resultierende Artefakt wird aus dem Verzeichnis dist heraus an den Endpunkt ausgeliefert. Die Sicherheitskonfiguration ist in netlify.toml zentral definiert und stellt sicher, dass alle Ressourcen unter /* standardmäßig gegen MIME-Typ-Missbrauch geschützt werden (X-Content-Type-Options: nosniff) sowie gegen Clickjacking (X-Frame-Options: DENY). Zudem wird die Referer-Policy auf strict-origin-when-cross-origin eingeschränkt und sensible Berechtigungen wie Kamera, Mikrofon oder Geolocation standardmäßig blockiert. Für statische Assets im Pfad /fonts/* ist eine aggressive Caching-Strategie (public, max-age=31536000, immutable) aktiviert, um die Ladezeiten zu optimieren. <!-- not-ev

Die Frontend-Struktur basiert auf einer klaren Trennung von Layouts und Komponenten, wobei BaseLayout und das spezialisierte LegalLayout als Gerüste für verschiedene Seiten dienen. Die verfügbaren Komponenten decken alle wesentlichen UI-Bereiche ab: Vom Header und Footer über spezifische Elemente wie ContactCTA, ContactForm, FloatingWhatsApp bis hin zu rechtlichen Sektionen (LegalSection) und Branding-Elementen (Logo, Icon). Die Seitenstruktur umfasst die Startseite (index), sowie dedizierte

Unterseiten für Leistungen, das Team, Über uns und die rechtlichen Texte (agb, datenschutz, impressum), ergänzt durch eine generische Fehlerseite (404).

Die Interaktion mit dem Backend erfolgt über einen spezifischen Endpunkt `/api/contact`. Dieser wird von der Frontend-Komponente `ContactForm` aufgerufen, wobei die Logik zur Verarbeitung in der Funktion `frontend/netlify/functions/contact.mjs` resides. Die Architektur erlaubt hier explizit nur POST-Anfragen und lehnt andere Methoden mit einem HTTP-Status 405 ab. Im Hintergrund wird dieser Endpunkt von einer Serverless-Funktion (`serverless_46`) genutzt, um über SMTP (vermittelt durch `Nodemailer`) eine E-Mail an das Backend zu versenden, was die Kommunikation zwischen der Frontend-Anwendung und dem Mail-Server sicherstellt. Zusätzlich zum Frontend existiert ein Python-basiertes Backend im Verzeichnis `backend`, das für weitere Server-Funktionen wie den Mailer (`mailer.py`) und den Hauptserver (`server.py`) zuständig ist, während Cron-Jobs in `.emergent/cron` automatisierte Webhook-Auslösungen verwalten.

Kurzreferenz

1234 Fakten / 280 Beziehungen · 2026-09-09T18:27:42.367Z.

